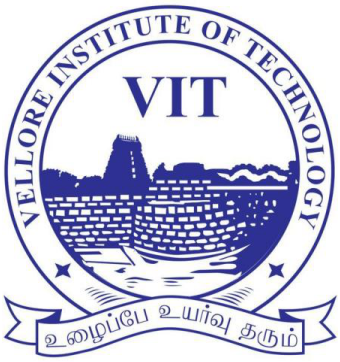
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT

CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset

Submitted by

Srikiran – 24BCE2215

Dhruv – 24BCE2211

Prince – 24BDS0153

Under the guidance of

Nithya K,

Assistant Professor Senior Grade II

SEPTEMBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project work titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** is a bonafide record of work carried out by **Srikiran (24BCE2215), Dhruv (24BCE2211), Prince (24BDS0153)**, submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: _____ :

Date: _____

DECLARATION

We hereby declare that the project report titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of Nithya K. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: _____

Date: _____

Signatures of all team members

ABSTRACT

CodeTrace is a static program-analysis platform for CT-C, a documented subset of C, and this report documents its Review 2 state. A hand-written lexer produces tokens with source locations; a grammar-driven parser generator builds canonical LR(1), LALR(1) or SLR(1) tables for the same grammar, and the resulting syntax tree is resolved by a scoped semantic analyser that assigns stable symbol identifiers. Those identifiers drive per-function control-flow graphs and a shared worklist framework that computes reaching definitions, liveness and definite assignment. A declarative rule language, parsed by the same LR machinery, declares taint sources, sinks, sanitizers and validators; the analysis follows explicit data flow through assignments, arguments and return values, and relates each result to a typed dependency graph and to forward, backward or finding-focused slices. A nine-view browser workbench and an HTTP API expose the compiler artefacts and the security conclusions together, and the report export is available as stable JSON or as a self-contained HTML file. The Review 2 build passes 350 engine tests, 53 interface tests, 69 independent audit checks and 18 of 18 injected-defect mutations, and reproduces its labelled corpus of 33 programs at precision 1.0 and recall 1.0. The contribution is inspectability: every reported flow can be traced back to the tokens, statements and analysis facts that produced it.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 6
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives & Novelty 7
CHAPTER 2 LITERATURE SURVEY 9
2.1 Existing Systems / Related Work 9
2.2 Research Gap 10
CHAPTER 3 SYSTEM DESIGN 12
3.1 Proposed System Architecture 12
3.2 Module Description 13
3.3 Compiler Phases Implementation 14
CHAPTER 4 IMPLEMENTATION DETAILS 16
4.1 Technology and Application Workflow 16
4.2 Repository and GitHub Link 16
4.3 Features Implemented for Review 2 17
4.4 Feature Walkthrough with Screenshots 18
4.5 Worked Example: Command Injection 30
4.6 Located Diagnostics and Partial Analysis 32
4.7 Work File Explanation 33
4.8 Module-Wise Reference 38
4.9 Verification, Testing and Results 41
REFERENCES 44

CHAPTER 1
INTRODUCTION

1.1 Background

A compiler front end converts source text into a structured representation that later stages can reason about. Lexical analysis groups characters into tokens, syntax analysis checks whether the token sequence follows a grammar, and semantic analysis resolves declarations and checks the relationships between names and operations. These steps are useful beyond translation: once a program is represented as an abstract syntax tree and its identifiers are resolved, an analysis can reason about how values move through statements and functions [1].

Static program analysis derives properties of a program without running the submitted program. Data-flow analysis evaluates facts at program points by following control flow, while taint analysis follows values from designated sources to operations that require particular handling. These methods are relevant to injection risks, where externally influenced data may reach an operation such as a command interpreter, a database query, a file path or a format-string function. The analysis must distinguish a call that merely exists from a call whose argument may be influenced by untrusted input [4, 5, 9].

CodeTrace applies these ideas to CT-C, a documented, deliberately small C-like language subset. The language front end, the intermediate representations and the analysis results are all made available for inspection through a browser interface and an HTTP API. The aim is to show how compiler-design components support a traceable security analysis, rather than to provide a full C compiler or to claim comprehensive vulnerability detection.

1.2 Problem Statement

Security findings are difficult to evaluate when a tool reports only a rule name and a source line. Without a model of syntax, symbol identity and control flow, it is difficult to explain whether a value can reach a sensitive operation, which assignments and function calls connect the two, or where an analysis assumption affects the conclusion. A second problem is interpreting an empty result: the absence of a reported flow is meaningful only in relation to the language constructs, rules and analysis approximations actually considered.

The project addresses the following problem: given a single source file written in the supported CT-C subset, construct the compiler representations needed for analysis, derive selected control-flow and data-flow facts, identify potential rule-defined source-to-sink flows, and present the reasoning and the limits behind each finding. The analysed source must remain data; it is not executed as part of the

analysis. For Review 2 the same problem is also applied to the tool itself: the artefacts must be reachable from the interface, and the claims made about the tool must be reproducible by the reader.

1.3 Objectives & Novelty

The main objectives of the project are to:

- implement a lexer that preserves source positions and provides located diagnostics for malformed and unsupported input;
- generate LR-family parsing tables from the CT-C grammar and build a source-located abstract syntax tree;
- resolve declarations with scoped symbol tables and report semantic issues that affect the reliability of later analysis;
- construct per-function control-flow graphs and compute reaching definitions, liveness and definite assignment with one shared worklist framework;
- specify taint sources, sinks, sanitizers, validators and flows in a declarative rule language, then track explicit data flow across statements and supported function calls;
- build typed dependency relationships and forward, backward and finding-focused slices that explain the statements related to a result;
- present findings through a browser workbench and an HTTP API, including the evidence path, certainty, known limitations and remediation guidance;
- expose the compiler artefacts themselves, so that a reader can move from a reported flow back to the token stream, the syntax tree, the control-flow graph and the data-flow facts that produced it.

Contribution and scope. The contribution is the integration of these established methods into a small, inspectable pipeline. A shared grammar-processing approach is used for CT-C and for the rule language, and later stages work from parsed structures and resolved symbols rather than searching raw source text for names. Findings are accompanied by the path and the assumptions used to justify them, so the reader can connect the security result to the compiler artefacts that support it.

The scope is intentionally bounded. CT-C supports selected scalar types, fixed-size array declarations, expressions, functions, conditionals and loops. Full C features such as pointer and alias analysis, preprocessor semantics and multi-file linking are outside the implemented language model. The resulting findings should therefore be read as potential flows within the supported subset and the configured rules, not as proof of exploitability or as a guarantee that a program is free of vulnerabilities.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing Systems / Related Work

Compiler construction provides the front-end methods used by CodeTrace. Standard compiler texts describe the progression from tokens to syntax trees and semantic representations [1]. Knuth established the LR approach to parsing from left to right [2], DeRemer's simple LR method gives a compact table-construction approach [3], and the LALR look-ahead computation introduced by DeRemer and Pennello [11] is what keeps the default table set small without losing the right-hand-context information that SLR lacks. CodeTrace applies an LR-family, table-driven parser to its own defined grammar; it does not rely on a general-purpose C compiler to produce the structures used by the analysis.

Data-flow analysis provides a formal way to compute program facts across a control-flow graph. Kildall's framework unified several global program optimisations using transfer functions and iteration [4], while Nielson, Nielson and Hankin give a systematic treatment of program analysis and its underlying lattices and fixed-point methods [5]. CodeTrace applies this general approach to reaching definitions, liveness and definite assignment. The results explain which definitions may reach a statement and which values may be used later.

Program slicing addresses program comprehension by selecting the statements relevant to a chosen value or computation. Weiser introduced slicing as a way to isolate statements related to a criterion [6]; dependence-graph methods model the data and control relationships explicitly [7], and interprocedural slicing extends the same idea across function boundaries [12]. Reps, Horwitz and Sagiv formulate a graph-reachability approach for precise interprocedural data-flow problems under defined assumptions [8]. CodeTrace uses a typed dependency graph to make selected relationships visible and computes slices around statements or findings.

Security-oriented static analysis uses program structure and rules to identify potential weaknesses without executing the target program. Practical tools show that a source-to-sink model can trace flows that simple function-name checks cannot, while remaining dependent on the language model, the source and sink definitions and the analysis approximations [9, 13]. Weakness catalogues such as CWE and the OWASP Top 10 provide the vocabulary in which those results are reported [10, 14]. CodeTrace adopts this source-to-sink model for a small language subset and makes the evidence path and the uncertainty part of the reported result.

Table 2.1: Approaches reviewed and their relevance to CodeTrace.

Approach	Typical strength	Relevance to CodeTrace	Source
Compiler construction	Formal recognition of syntax and production of structured program representations.	Supplies the lexer, LR parser, AST and semantic foundation for the analysis.	[1], [2], [3]
Data-flow frameworks	Systematic computation of facts over control-flow graphs.	Supports reaching definitions, liveness and definite assignment.	[4], [5]
Program slicing and dependence graphs	Identifies statements related through data or control dependencies.	Provides the explanation and finding-focused views of connected statements.	[6], [7], [12]
Interprocedural analysis	Reasons across function boundaries with explicit assumptions.	Motivates the function summaries used for argument and return flows.	[8], [12]
Security-oriented static analysis	Checks whether data from configured sources may reach sensitive operations.	Forms the basis of the rule-driven findings and of the remediation guidance.	[9], [10], [13], [14]

2.2 Research Gap

The gap addressed by CodeTrace is primarily one of integration and inspectability in a Compiler Design learning context. Textbooks explain individual algorithms, and security analysers present their conclusions through findings and rule identifiers, but a learner often needs to connect several stages to understand why a result appears. CodeTrace places the lexer and parser, the syntax tree, the scoped symbols, the control-flow graph, the data-flow facts, taint propagation and the dependency relationships in a single workflow over the same source program.

A second design concern is the interpretation of negative results. A list with no findings does not establish that the source is safe. The tool therefore records which constructs are unsupported, marks an analysis as partial when diagnostics or limits affect the result, and exposes the reasons why inspected sink calls were not reported. This makes the boundary of the analysis visible instead of allowing a quiet result to be mistaken for a complete proof.

A third concern is explanation. A useful security finding should state the source and the sink, show the sequence of data-flow steps, identify the relevant sanitizers or validator assumptions, describe its certainty, and state the limitations that affect its interpretation. The typed dependency graph and the program slice support a second view of the same result: the evidence path describes the flow, while the graph and the slice show the related statements and relationships.

Table 2.2: Design concerns, the CodeTrace response and the boundary that remains.

Design concern	CodeTrace response	Boundary
Inspectability	Token, syntax, graph, data-flow, slice and evidence views over one run.	The views reflect only the implemented model.
Negative results	Unsupported input, partial-analysis reasons and unreported-sink dispositions are shown where available.	No finding is not a proof of safety.
Explainability	Source and sink details, ordered flow steps, certainty and remediation travel with each finding.	The reported path may be approximate.
Learning value	One source representation is reused across the compiler and the analysis stages.	The supported language is deliberately small.
Reproducibility	Every claim in this report is measured by a script in `scripts/`.	The corpus is authored by the project team, not by an external body.

Positioning of the proposed system. CodeTrace is not intended to replace mature static application security testing tools or to claim greater detection accuracy than them. Its contribution is that the underlying compiler and analysis representations are available for inspection, and that the implementation runs without third-party runtime packages or a separate build step. This makes the relationship between compiler construction, data-flow analysis and security reasoning directly observable in an academic project. The research gap is therefore addressed within a constrained language, and that same constraint is a limitation: the system does not model all constructs found in production C, and its interprocedural summaries and array treatment remain approximate.

CHAPTER 3
SYSTEM DESIGN

3.1 Proposed System Architecture

CodeTrace separates the browser interface from the analysis engine. The browser submits source text and options to the Python HTTP API. The API validates the request and invokes the analysis pipeline; the engine returns a program object containing the intermediate representations, diagnostics, data-flow results, taint facts, the dependency graph, slices and findings. The API exposes those results to the interface and to the JSON and HTML reporting modules.

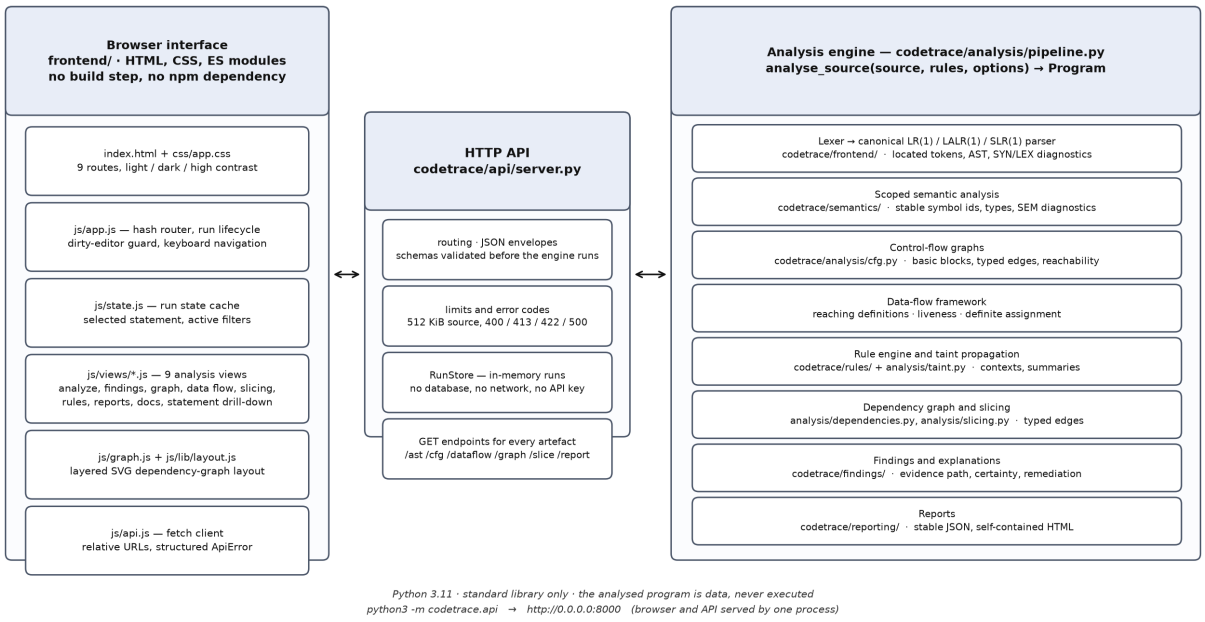


Figure 3.1: Runtime architecture: browser workbench, HTTP API and analysis engine in one process.

There are exactly two processes at run time: the Python server and the browser. The server is started with `python3 -m codetrace.api`, serves the interface and the API from the same origin, and keeps recent runs in memory. No build step, bundler, database, API key or network access is involved. The browser is a presentation layer rather than a second analysis engine: it renders the structured output it receives and requests detail for a run, and it never re-derives an analysis fact of its own.

The analysis pipeline itself advances through lexical analysis, parsing, semantic analysis and construction of a control-flow graph. Data-flow analysis supplies the facts used by taint propagation. A typed dependency graph is then built to represent data, control, call, return, definition, sanitisation and source-to-sink relationships. Slicing and finding explanation consume those relationships before the

reporting stage produces a stable, reviewable result.

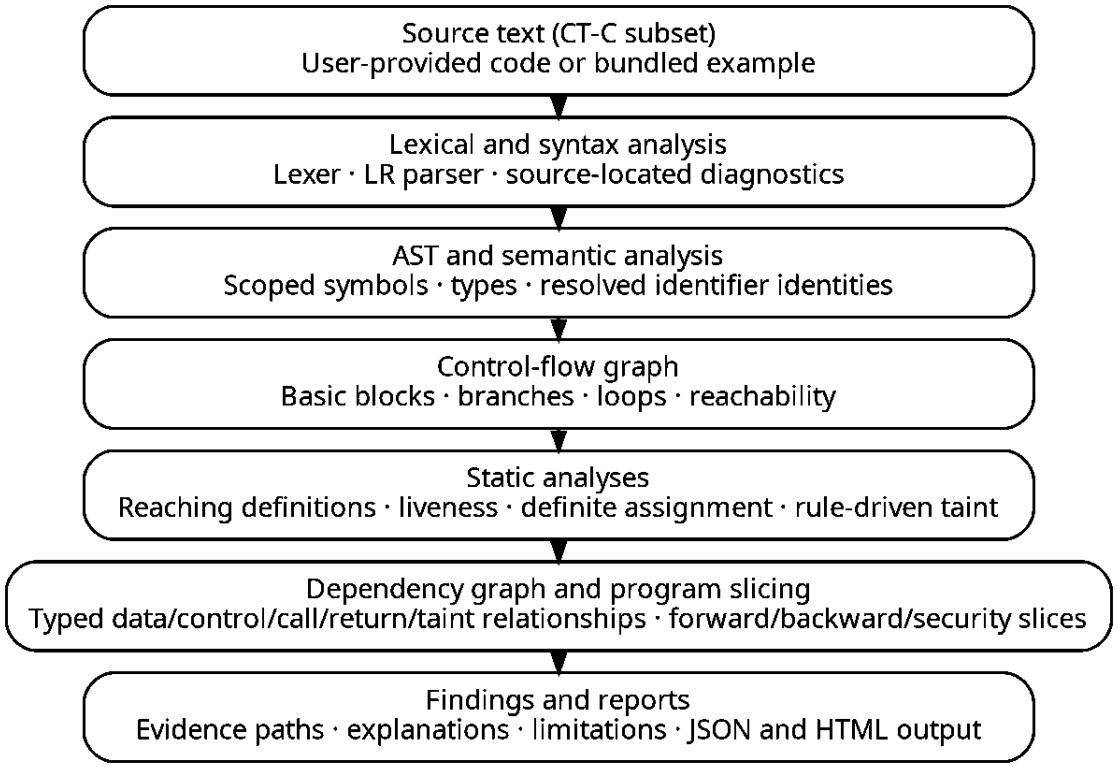


Figure 3.2: Analysis pipeline: each stage consumes the artefacts of the stage above it.

3.2 Module Description

The implementation is divided into packages with defined responsibilities. These boundaries allow each stage to operate on explicit data structures and give the interface a consistent representation of an analysis run.

Table 3.1: Module responsibilities and their main outputs.

Module	Responsibility	Main output
Language front end `codetrace/frontend`	Tokenisation, grammar analysis, LR-family tables, parser and AST node definitions.	Located tokens, syntax tree and syntax diagnostics.
Semantic analysis `codetrace/semantics`	Scope handling, stable symbol identities, type and declaration checks, per-statement reads and writes.	Resolved symbols and semantic diagnostics.
Control flow and data flow `codetrace/analysis`	Per-function basic blocks, typed edges, reachability and worklist analyses.	CFGs, definitions, reaching-definition, liveness and definite-assignment states.
Rules and taint analysis `codetrace/rules`, `analysis/taint.py`	Declarative rule parsing; source, sink, sanitizer, validator and flow matching.	Taint paths, function summaries, dispositions and certainty information.
Dependencies and slicing `analysis/dependencies.py`, `analysis/slicing.py`	Typed dependence edges and statement reachability for a selected criterion.	Program dependency graph and forward, backward or finding-focused slices.
Findings, reporting and API `codetrace/findings`, `codetrace/reporting`, `codetrace/api`	Explanations, stable finding identity, JSON and HTML serialisation, HTTP routing and request validation.	Interactive results and exported reports.
Interface `frontend/`	Nine views over one run, graph layout, filters, theme handling.	The workbench that the screenshots in Chapter 4 show.

3.3 Compiler Phases Implementation

Lexical analysis. The lexer converts source characters into typed tokens with offsets and line-and-column positions. It skips comments, recognises the supported identifiers, literals, punctuation and operators, and decodes escape sequences including `\xHH`. Malformed literals and constructs outside the CT-C subset receive explicit diagnostics, for example `LEX100` for a `switch` statement and `LEX101` for a preprocessor directive.

Syntax analysis and AST construction. A grammar-driven LR-family parser recognises the supported declarations, statements and expressions. The parser modes expose canonical LR(1), LALR(1) and SLR(1) table construction over the same 102-production grammar. The parser produces located syntax diagnostics together with panic-mode recovery and an expected-token list; accepted input is represented as an abstract syntax tree whose nodes carry half-open source spans.

Table 3.2: The same grammar, measured in each parser mode on this checkout.

Parser mode	States	Productions	Conflicts	Build time
LALR(1) — default	167	102	0	0.78 s
Canonical LR(1)	656	102	0	0.87 s
SLR(1)	167	102	0	0.06 s

Measured with `codetrace.frontend.pipeline.parser_tables`*; the canonical automaton is built before the LALR states are merged, which is why it has 656 states against 167.*

Semantic analysis and intermediate representation. Nested symbol tables distinguish declarations with the same spelling in different scopes and assign every declaration a stable integer identifier, so a later

stage refers to a symbol rather than to a name. Semantic checks identify unresolved names (SEM001), incompatible operations (SEM005), wrong call arity (SEM009) and shadowing (SEM018). The AST is then converted into per-function control-flow graphs whose basic blocks and typed edges model sequencing, branches, loops and returns.

Data-flow, taint and dependency analysis. A shared worklist framework computes reaching definitions (forward, may), liveness (backward, may) and definite assignment (forward, must) with bit-vector domains. Rule-driven taint analysis follows explicit data flow through assignments, expressions, arguments and return values, keeping a separate atom per context so that escaping a value for SQL does not clear its command context. The dependency graph records the reason for every data, control, call, return, definition, sanitisation and source-to-sink edge; slices use those edges to isolate the statements relevant to a chosen statement or finding.

Table 3.3: Compiler phases, the artefact each phase emits and where it is implemented.

Phase	Artefact produced	Implementation	Diagnostics
Lexical analysis	Located token stream, lexer statistics	`frontend/lexer.py`, `frontend/tokens.py`	`LEX001`–`LEX008`, `LEX100`–`LEX102`
Syntax analysis	Parse tree, canonical/LALR/SLR tables, recovery information	`frontend/grammar.py`, `frontend/clr_parser.py`	`SYN001`, `SYN010`, `SYN100`
AST construction	Typed nodes with half-open spans	`frontend/ast_nodes.py`	—
Semantic analysis	Scopes, symbols, call sites, accesses	`semantics/analyzer.py`, `symbol_table.py`	`SEM001`–`SEM901`
Control-flow graph	Basic blocks, typed edges, loops, reachability	`analysis/cfg.py`	`CFG001`
Data-flow analysis	IN/OUT sets, per-statement states, uninitialised reads	`analysis/dataflow.py` and the three problems	`DFA001`
Taint analysis	Taint atoms, sink hits, summaries, dispositions	`analysis/taint.py`	reported as findings
Dependency graph	Typed nodes and edges with reasons	`analysis/dependencies.py`	—
Slicing	Statement sets, inclusion reasons, subgraph	`analysis/slicing.py`	—
Findings and reports	Stable finding identifiers, JSON and HTML reports	`findings/`, `reporting/`	—

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 Technology and Application Workflow

The analysis engine is implemented in Python 3.11 using only the standard library. The browser application uses HTML, CSS and plain JavaScript ES modules and has no separate build step and no third-party runtime package. A local HTTP server, built on the standard library's `http.server`, exposes the analysis operations and the structured result data.

Table 4.1: Technology choices and the reason for each.

Layer	Technology	Why this choice
Lexer, parser, analyses	Python 3.11, standard library only	No installation step for the examiner; the algorithms are readable and auditable.
Parser generator	Canonical LR(1) with LALR(1) state merging and SLR(1) mode	One grammar feeds three table constructions, so the theory is visible and comparable.
Rule language	Declarative text parsed by the same LR machinery	Rules are validated with `line:col` diagnostics before any analysis runs.
Interface	HTML, CSS, native ES modules	No bundler, no npm dependency, and the module boundaries stay explicit.
Graph rendering	Hand-built layered SVG layout	Deterministic positions, keyboard-reachable nodes, no layout library.
Transport	JSON over HTTP on one origin	The interface uses relative URLs, so it also works behind a forwarded port.
Persistence	In-memory run store	The tool is local and stateless; a database would add deployment cost only.
Report export	Stable JSON and a self-contained HTML file	The report can be reviewed offline, and all analysed text is escaped.

A typical session begins when the user enters CT-C source code or selects a bundled example. The interface submits the source, the file name, the parser mode and the rule options. The server validates the request and invokes the analysis pipeline. The interface then presents the diagnostics, a run summary, the findings and the supporting views such as the program graph, the data-flow states and the slices. A user can inspect a finding, follow its evidence path and export the result as JSON or HTML.

4.2 Repository and GitHub Link

The complete implementation, the tests, the labelled corpus and the screenshots in this report are in a single public repository. The Review 2 state documented here is the state of the default branch at the time of submission.

- **Repository:** https://github.com/lostpilot404/CD-Project
- **Main branch:** `main`; the Review 2 material is also carried on the working branch.
- **Runtime requirement:** Python 3.11 or newer; no third-party package.
- **Start-up:** `python3 -m codetrace.api` then open `http://localhost:8000`.

- **Checks:** `scripts/tests.sh`, `scripts/evaluate.sh`, `python3 scripts/audit.py`, `python3 scripts/mutation_check.py`.

Table 4.2: Where everything lives in the repository (file and line counts measured from this tree).

Directory	Files	Lines	Contents
`codetrace/frontend/`	9	3,277	Lexer, grammar, LR table generators, parser, AST.
`codetrace/semantics/`	4	1,352	Scopes, symbol table, type checks, externals.
`codetrace/analysis/`	11	4,130	CFG, data-flow framework, the three problems, taint, dependencies, slicing, pipeline.
`codetrace/rules/`	4	1,235	Rule model, rule-language parser, built-in rule loader and `standard.ctrule`.
`codetrace/findings/`	3	1,225	Finding model, stable identity, run comparison, explanation.
`codetrace/reporting/`	3	650	JSON report and self-contained HTML report.
`codetrace/api/`	4	1,152	HTTP server, routing, validation, run store, schema catalogue.
`frontend/`	26	6,446	Shell, nine views, state, graph layout, API client, CSS.
`frontend/tests/`	5	2,115	Stub DOM, harness and interface tests against a real engine.
`tests/`	18	6,212	Engine, API, reporting, audit and end-to-end suites.
`evaluation/`	2	754	Corpus harness and hand-written expectations.
`scripts/`	11	5,890	Server, tests, evaluation, audit, mutation check and report builders.
`examples/`	34	307	33 labelled CT-C programs and one example rule file.
`docs/`	13	4,050	The documentation set listed in `README.md`.

Total across the directories above: 145 files, 36,871 lines, including `README.md` *and the report-building scripts but excluding the binary screenshot and PDF assets.*

4.3 Features Implemented for Review 2

Review 2 completes the analysis pipeline and the workbench that exposes it. The features below are implemented, covered by tests, and visible in the screenshots that follow. Section 4.4 takes each feature in turn and shows the running application.

Table 4.3: Features added for Review 2, where they live and how they are verified.

Feature	Where it lives	Shown in	Verified by
Overview dashboard and run summary	`frontend/js/views/overview.js`	Section 4.4.1	`tests/test_api.py`
Analyze Code workbench with parser-mode and rule options	`frontend/js/views/analyze.js`	Section 4.4.2	`test_api`, `views.test.js`
Live dirty-editor alerting	`frontend/js/app.js`, `state.js`	Section 4.4.3	`regressions.test.js`
Located syntax diagnostics and partial-analysis status	`frontend/lexer.py`, `clr_parser.py`	Section 4.4.4	`test_lexer`, `test_parser`
Findings list with the ordered evidence path	`views/findings.js`, `findings/explain.py`	Section 4.4.5	`test_workflow`, `test_taint`
Interactive typed program dependency graph	`frontend/js/graph.js`, `analysis/dependencies.py`	Section 4.4.6	`views.test.js`, `test_slicing`
Data-flow inspector with per-block and per-statement states	`views/dataflow.js`, `analysis/dataflow.py`	Section 4.4.7	`test_dataflow`
Program slicing with inclusion reasons	`views/slicing.js`, `analysis/slicing.py`	Section 4.4.8	`test_slicing`
Declarative rule editor with validation	`views/rules.js`, `rules/rule_language.py`	Section 4.4.9	`test_rules`
Report export and run comparison	`views/reports.js`, `reporting/`	Section 4.4.10	`test_reporting`
Documentation view generated from the grammar	`views/docs.js`, `api/server.py`	Section 4.4.11	`test_api`
Dark and high-contrast themes	`css/app.css`, `app.js`	Section 4.4.12	`views.test.js`

4.4 Feature Walkthrough with Screenshots

Each figure below is a screenshot of the running application, captured from the local server with the bundled `command_injection.ctc` example or, where the feature needs different input, from the example that exercises it. The header of every page shows the engine version, the rule-set fingerprint and the grammar size, so a reader can tell which build produced the picture. Each screenshot is reproduced here at the full width of the text block; the unmodified 2049 x 1260 captures are also stored in `docs/review2_screenshots/` for reading at full size.

4.4.1 Overview dashboard and run summary

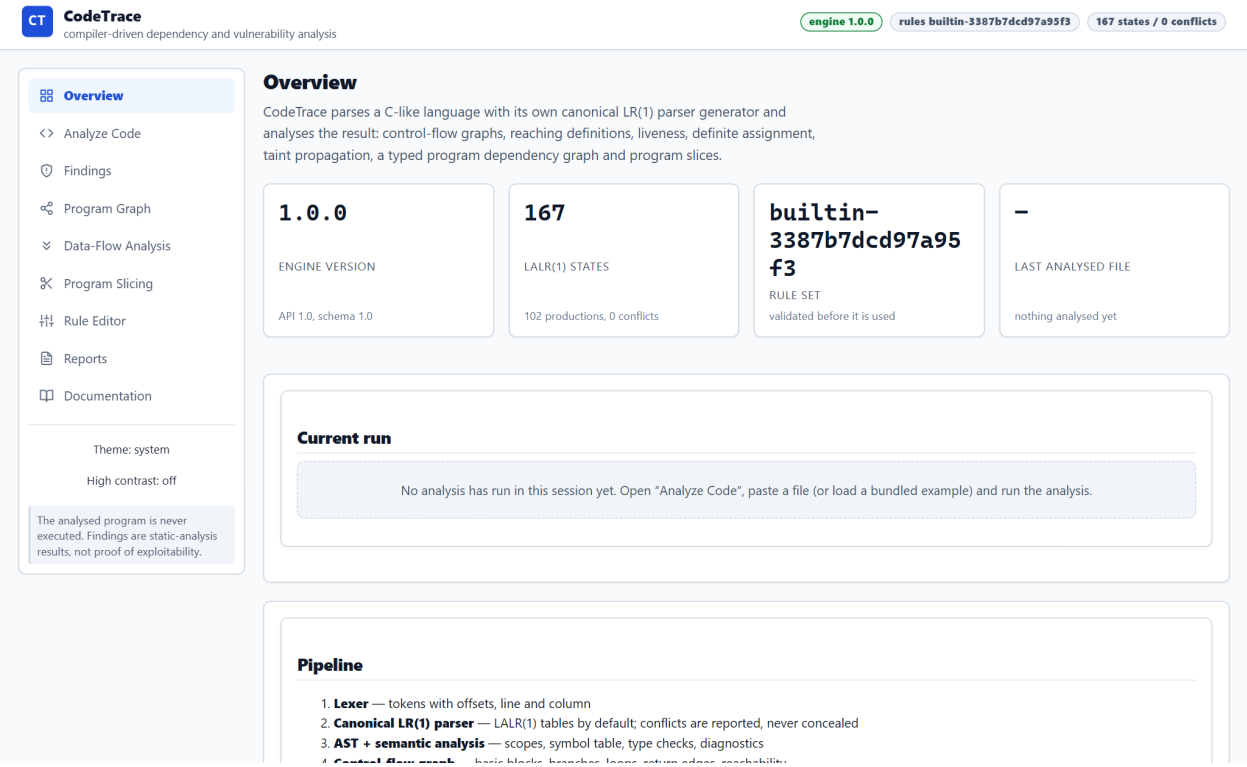


Figure 4.1: Overview dashboard and run summary — screenshot of the running application.

The landing view states what the tool is and what it currently holds: the engine version, the resolved rule-set fingerprint, the grammar size (167 LALR(1) states over 102 productions, with zero conflicts) and the last analysed file. The run panel repeats the same summary that the API returns, so the dashboard is a view of data rather than a separate computation.

- Metric cards are read from `/api/health` and `/api/runs/{id}`; nothing is hard-coded.
- An empty state explains how to start, instead of showing zeros without context.
- The pipeline list underneath names the nine stages and their artefacts.

4.4.2 Analyze Code workbench

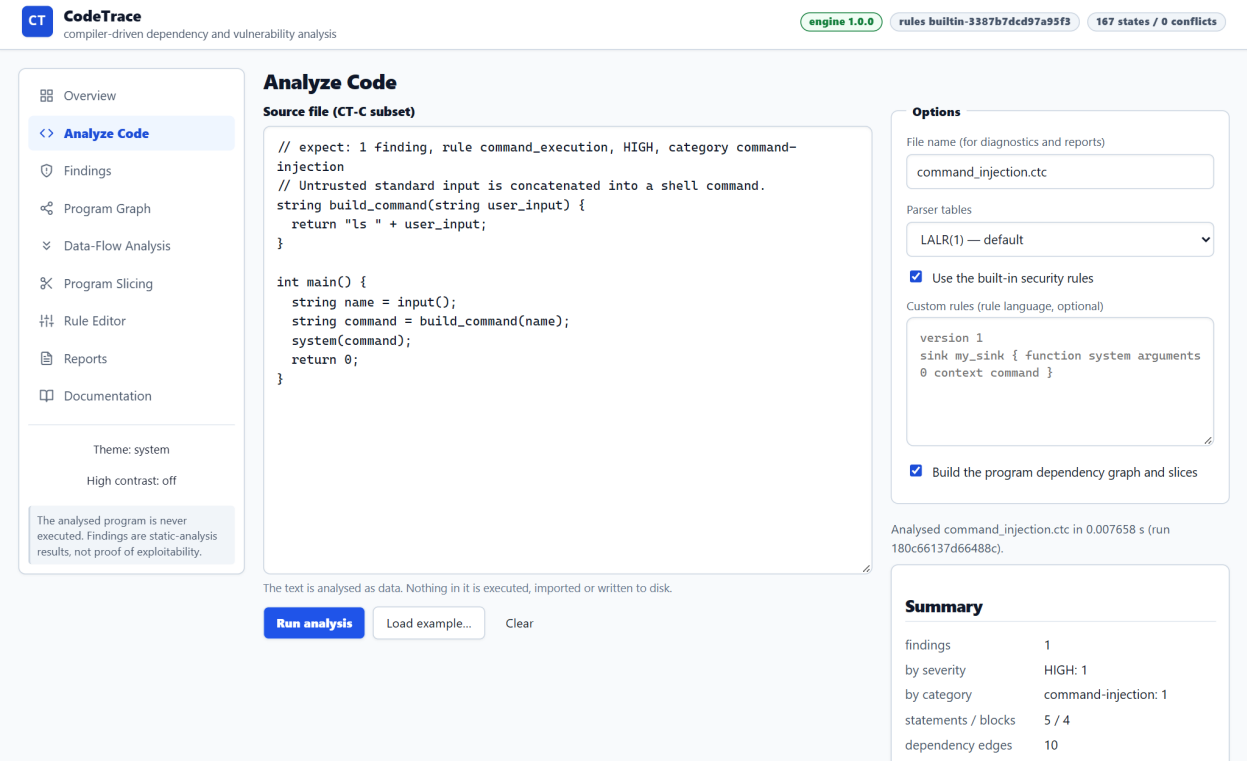


Figure 4.2: Analyze Code workbench — screenshot of the running application.

The workbench is the entry point for an analysis. The editor holds the CT-C source, and the options panel next to it sets the display file name, the parser mode (LALR(1), SLR(1) or canonical LR(1)), whether the built-in rules apply, an optional custom rule file and whether the dependency graph and slices should be built. The summary panel updates from the run that the server returns.

- The analysed text is never executed: it is tokenised, parsed and inspected as data.
- Skipping the dependency graph is an explicit trade-off, not a hidden failure: `/graph` and `/slice` then answer `409` with a reason.
- Every request is validated before the engine runs; the error envelope names the field.

4.4.3 Live dirty-editor alerting

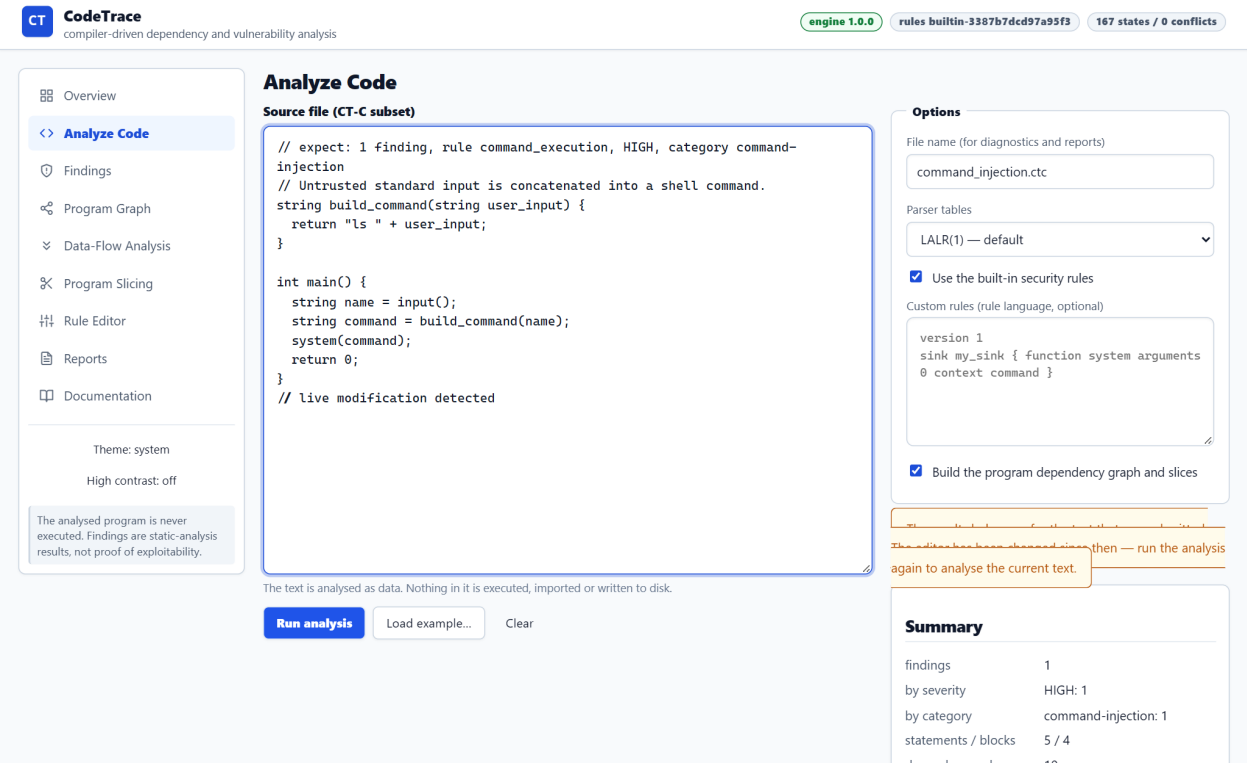


Figure 4.3: Live dirty-editor alerting — screenshot of the running application.

Analysis results describe the text that was submitted. When the editor changes afterwards, the results on screen are stale, and a reader can easily attribute an old finding to new code. The workbench tracks the submitted text against the current buffer and raises a warning as soon as they differ, asking for a fresh run before the results are cited.

- The comparison is on the exact submitted string, so any edit, including whitespace, clears the result's validity.
- The warning is assertive but does not hide the previous run; the reader decides when to re-run.
- Findings, graph and slices all share one run identifier, so a stale state is visible in every view at once.

4.4.4 Located syntax diagnostics and partial analysis

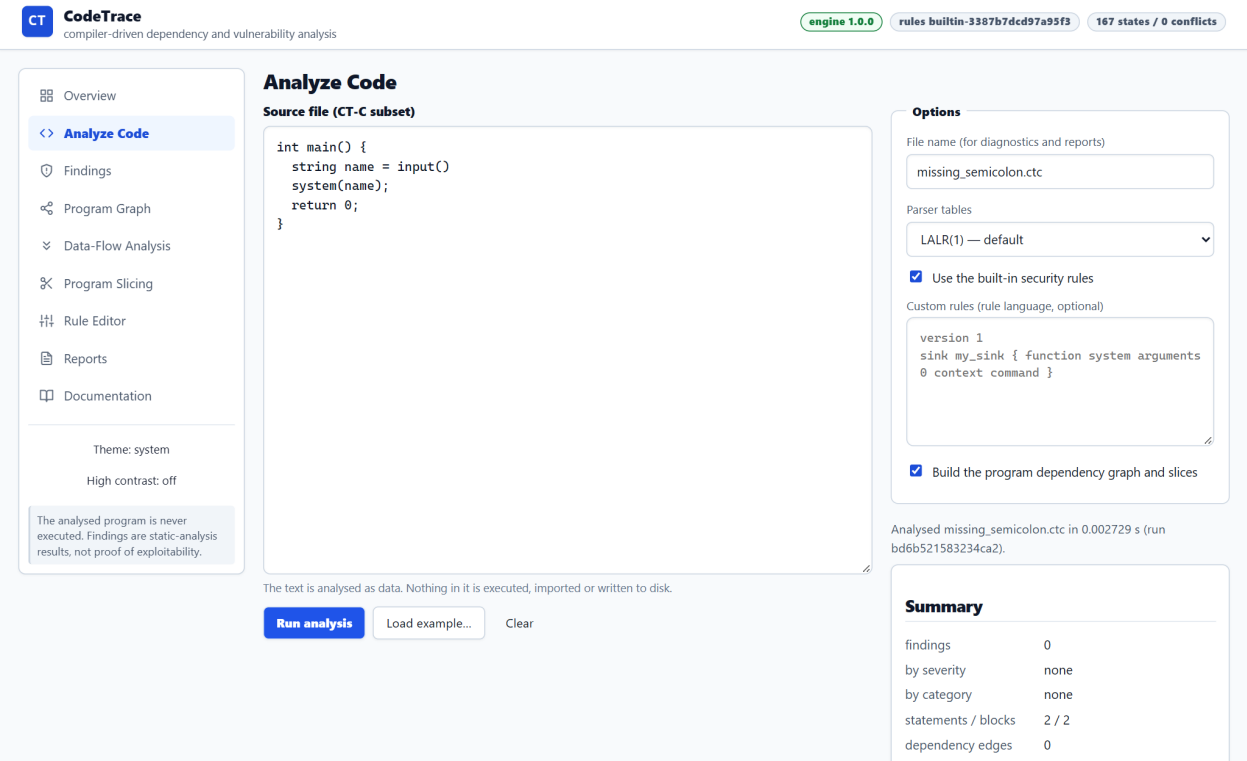


Figure 4.4: Located syntax diagnostics and partial analysis — screenshot of the running application.

A file that does not parse still produces a useful result. The missing-semicolon example is rejected by the parser, the diagnostic is reported with its `line:column` position and an expected-token list, and the run is marked partial: no findings are claimed, and the summary says so rather than reporting a clean analysis.

- Panic-mode recovery keeps the parser moving, so one error does not hide the next.
- Unsupported constructs are diagnosed too: `LEX100` for `switch`, `LEX101` for `#include`, `SYN100` for a pointer type.
- A partial run is never presented as evidence that the program is safe.

4.4.5 Findings and the ordered evidence path

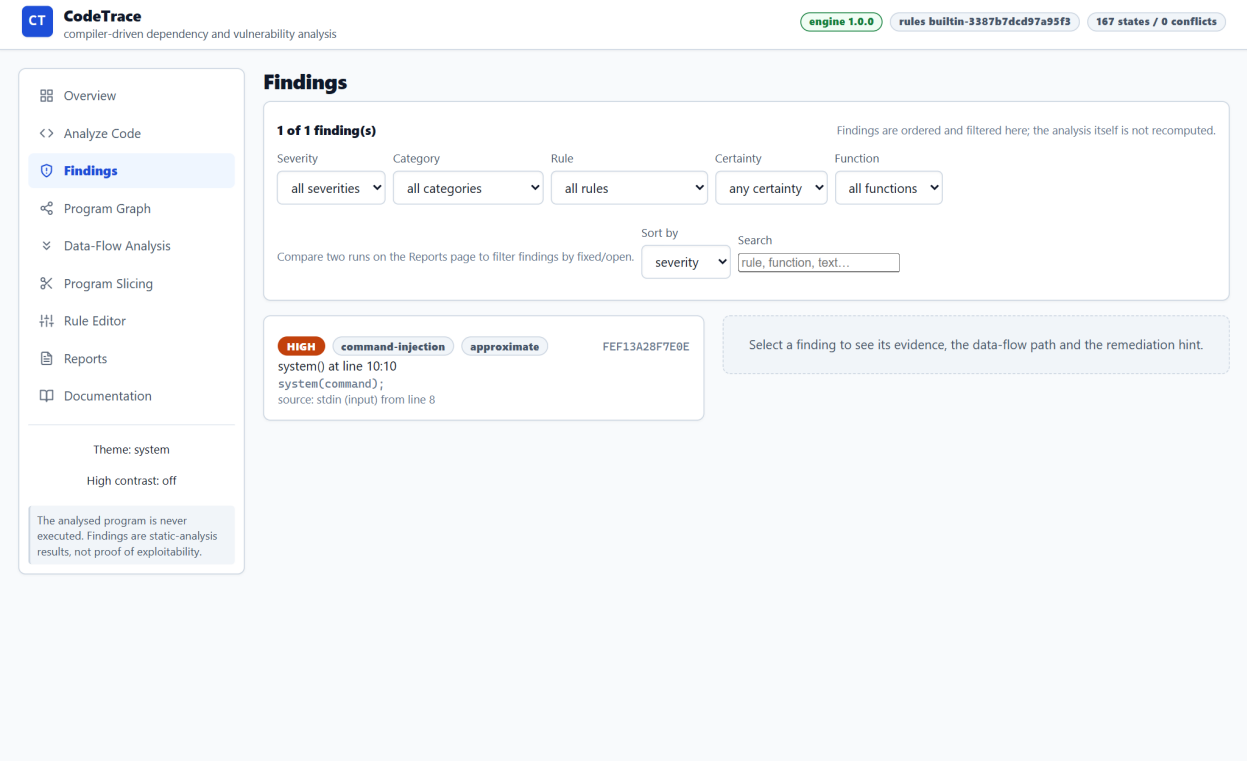


Figure 4.5: Findings and the ordered evidence path — screenshot of the running application.

Each finding carries a stable identifier, a rule, a category, a severity and a certainty. Selecting one shows the evidence path: the source call, the declarations it reaches, the argument that crosses a function boundary, the return that brings the value back, and the sink that consumes it. Filters and the free-text search operate on the findings of the current run and never re-run the analysis.

- The identifier is derived from the rule, the sink operation and the statement text, so inserting a line above a finding does not make it look new.
- Certainty is stated explicitly; an interprocedural flow is reported as approximate.
- Limitations and remediation guidance travel with the finding into both report formats.

4.4.6 Interactive program dependency graph

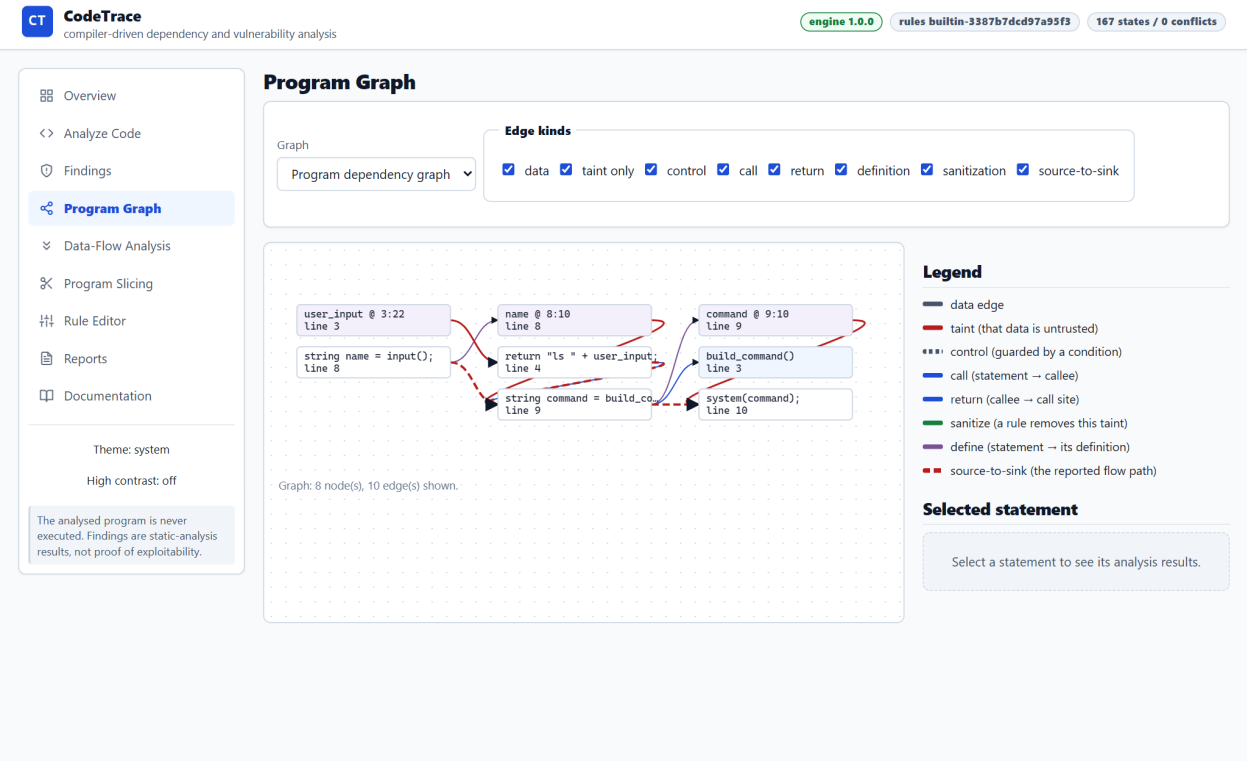


Figure 4.6: Interactive program dependency graph — screenshot of the running application.

The graph view renders the typed dependency graph for the run: data, control, call, return, definition, sanitisation, taint and source-to-sink edges, each drawn with its own style and each selectable. Selecting a node opens the statement inspector, which lists the reaching definitions, live variables and taint atoms at that statement.

- The layout is computed by a layered pass over the edges, so the same run always draws the same picture.
- Edge kinds can be filtered, which is how the reported flow is separated from the supporting dependencies.
- The view states how many nodes and edges are shown out of the whole graph.

4.4.7 Data-flow analysis inspector

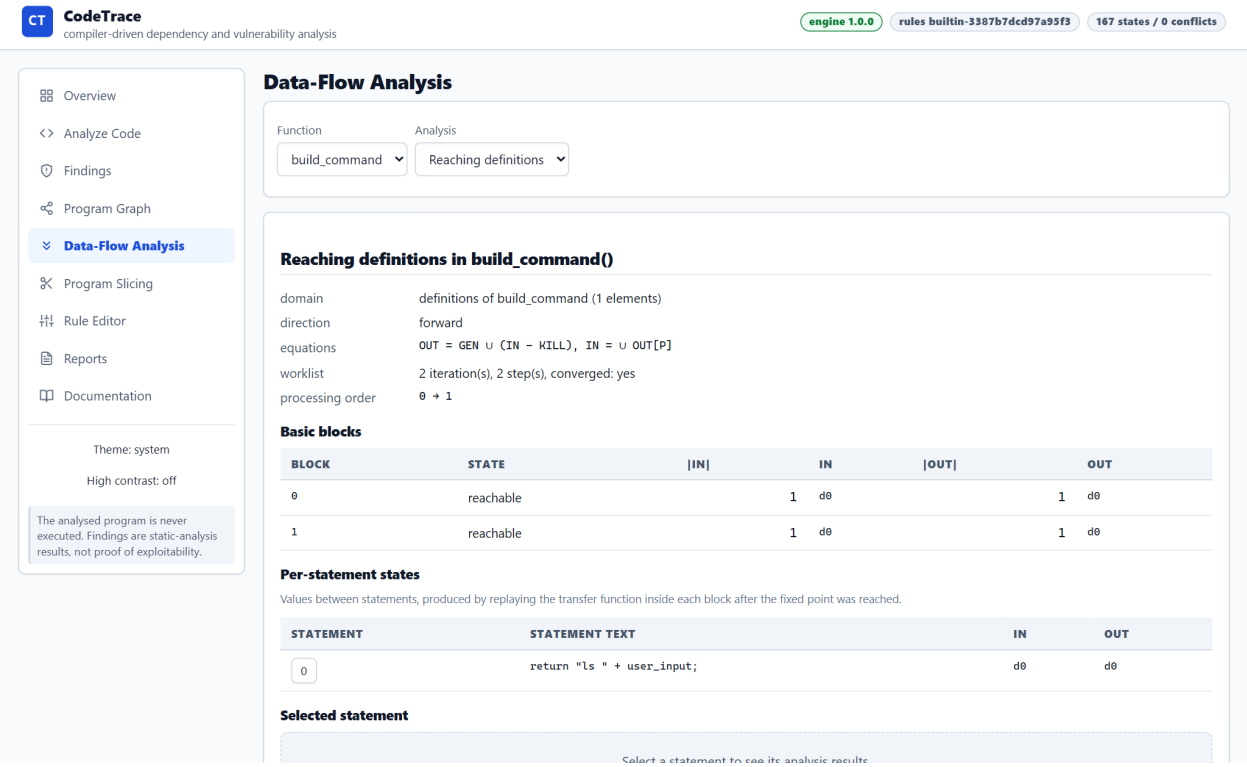


Figure 4.7: Data-flow analysis inspector — screenshot of the running application.

For each function the inspector shows one of the three data-flow problems with its direction, domain, equations, the number of worklist iterations and the processing order, then the IN and OUT set of every basic block and the state between statements. Nothing here is illustrative: the numbers come from the fixed-point solver that the tests exercise.

- The equations are printed next to the result, so the transfer function can be checked against the lattice it belongs to.
- Per-statement states are produced by replaying the transfer function inside each block after the fixed point is reached.
- Definite assignment additionally reports the reads that may happen before a write (`DFA001`).

4.4.8 Program slicing with inclusion reasons

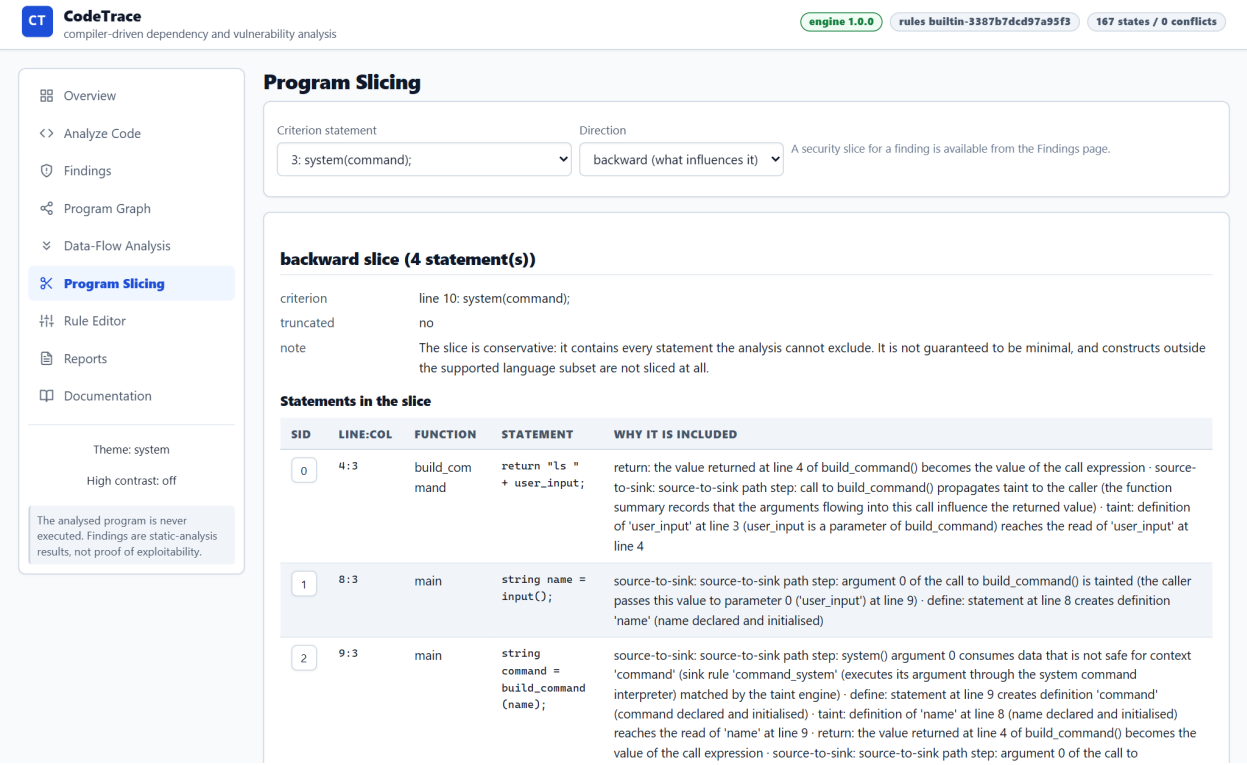


Figure 4.8: Program slicing with inclusion reasons — screenshot of the running application.

A slice answers a narrower question than the graph: which statements can influence this one, or be influenced by it. The view reports how many statements are in the slice and, for each one, the edge or fact that put it there. The same machinery produces the security slice reachable from a finding.

- Backward, forward and finding-focused directions share one reachability routine.
- The note states that the slice is conservative: it contains every statement the analysis cannot exclude, and it is not guaranteed to be minimal.
- Constructs outside the supported subset are not sliced, and the note says so instead of returning a short slice silently.

4.4.9 Declarative rule editor

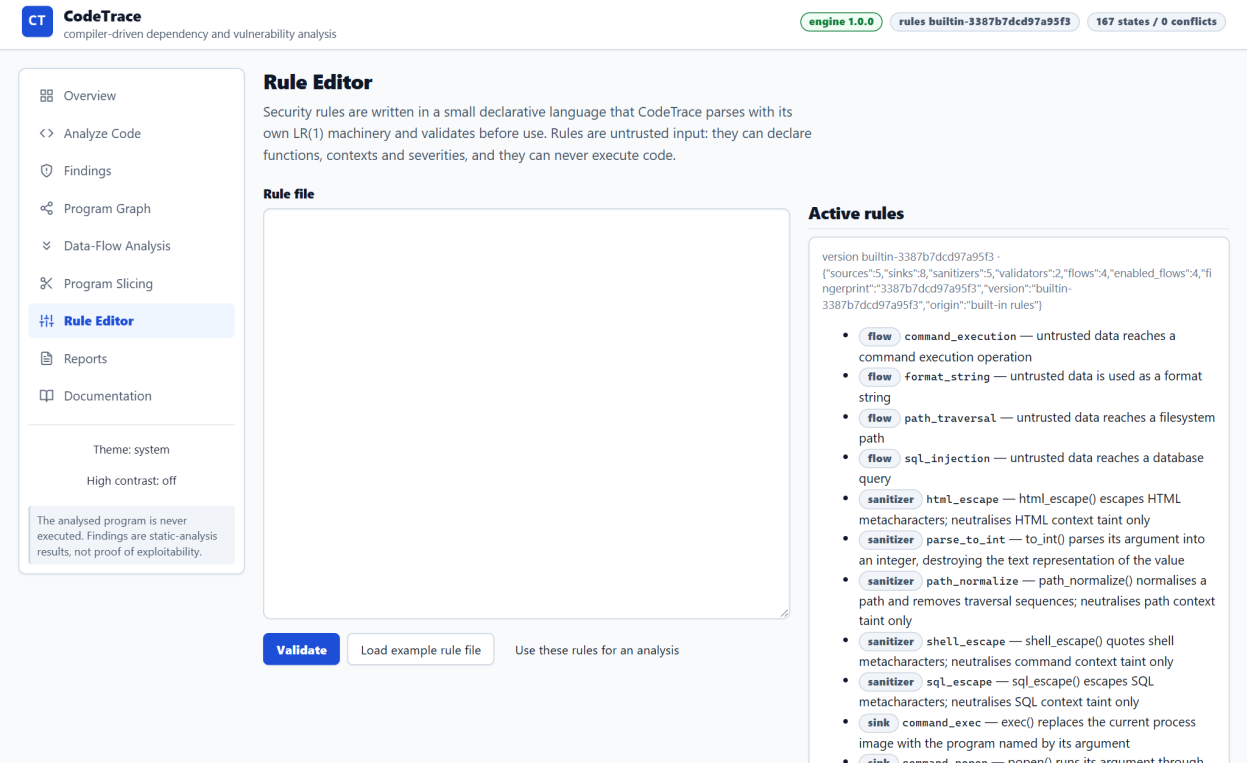


Figure 4.9: Declarative rule editor — screenshot of the running application.

Security rules are written in a small declarative language that CodeTrace parses with the same LR machinery as CT-C. The editor validates a rule file and reports diagnostics with positions before the rules can be used; the panel on the right lists the rules that are actually active, with the fingerprint of the set.

- A rule file is untrusted input: it can declare functions, contexts and severities, and it can never execute code.
- Validation happens before analysis; an invalid file is rejected with `422` and a `line:col` diagnostic per problem.
- The built-in set is itself a rule file (`rules/builtin/standard.ctrule`), not code hidden in the engine.

4.4.10 Report export and run comparison

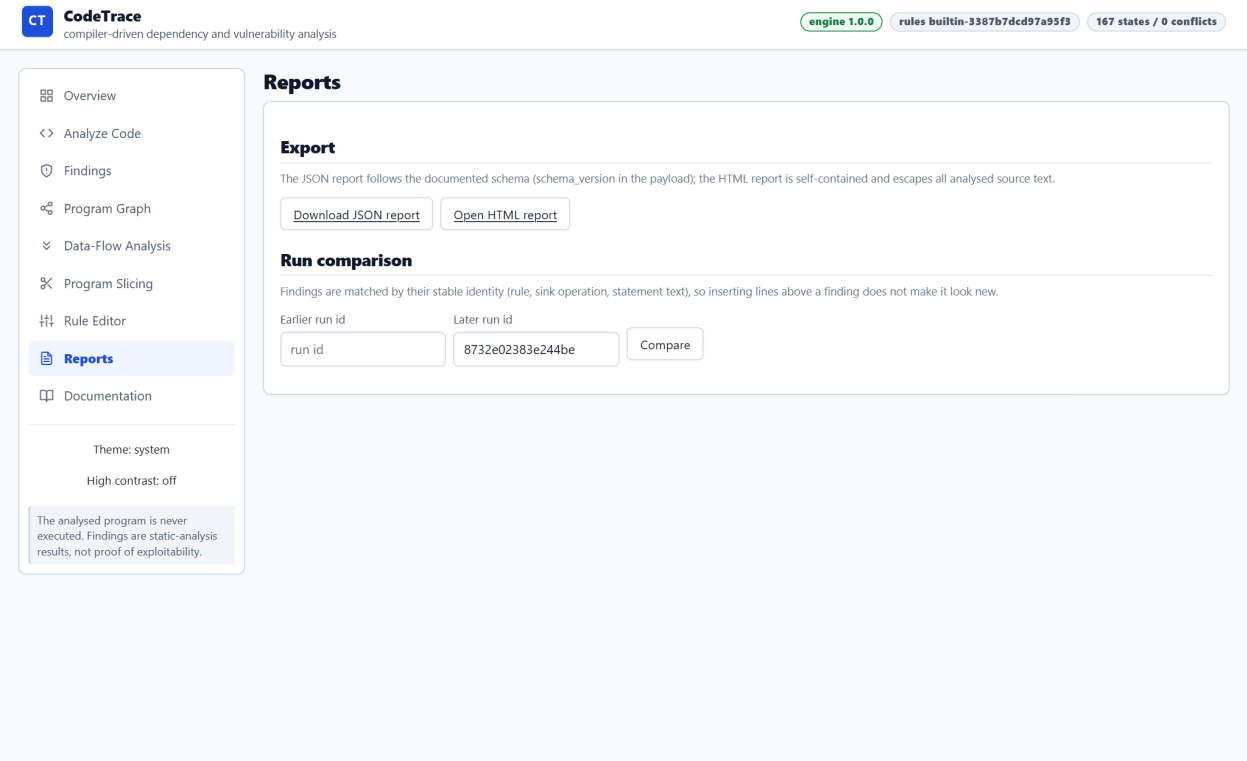


Figure 4.10: Report export and run comparison — screenshot of the running application.

A run can be exported as JSON that follows the documented schema, or as a single self-contained HTML file that opens without the server and escapes every piece of analysed text. The same page compares two runs: findings are matched by stable identity, so a finding that survives an edit is reported as unchanged rather than as new.

- Reports are generated on demand and never stored, so they cannot drift from the run.
- The JSON report has a documented key set and a schema version; it is the format the evaluation harness reads.
- The HTML report is one file with inline CSS, no script and no external request.

4.4.11 Documentation view and the language subset

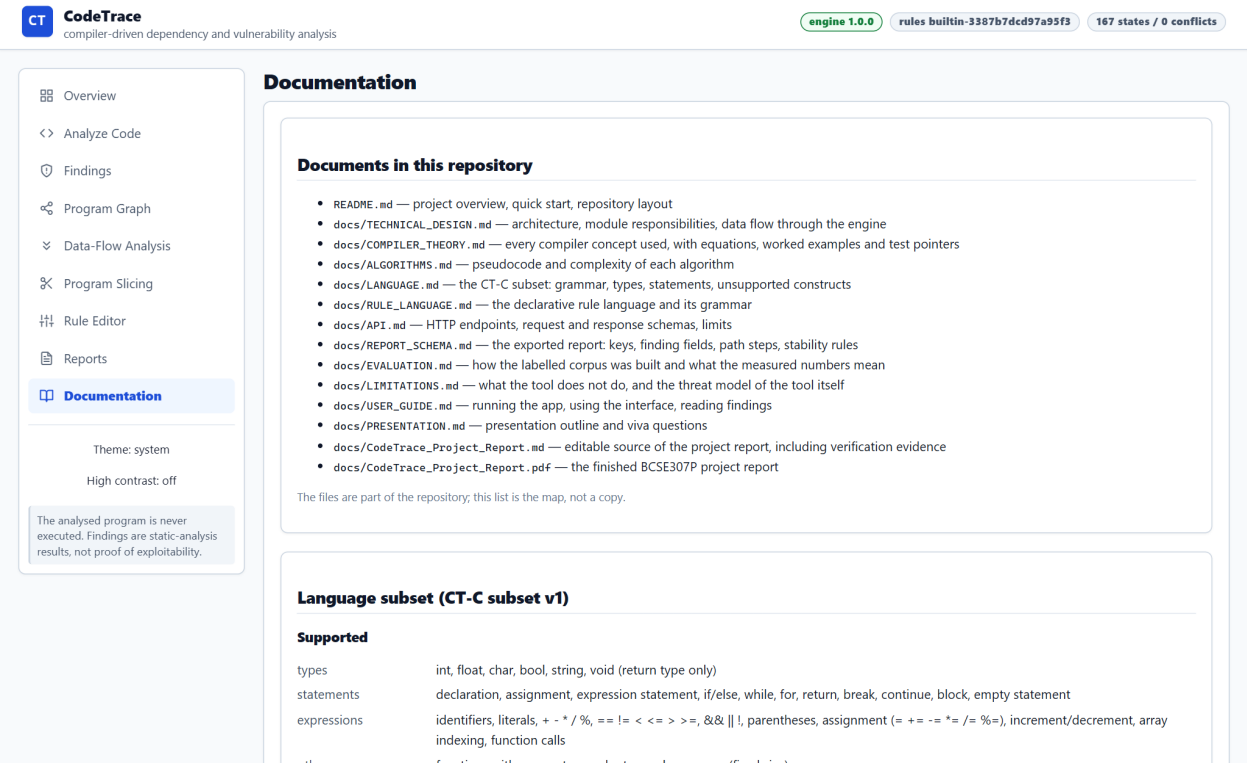


Figure 4.11: Documentation view and the language subset — screenshot of the running application.

The Documentation view maps the repository's documents and states the supported language subset: the types, statements, expressions and external functions, and the constructs that are explicitly rejected. The grammar and language descriptions are served by the API, so the page cannot drift from the implementation.

- `/api/language` and `/api/grammar` return the same data the parser uses.
- The unsupported list is part of the contract, not an omission in the documentation.
- The page links each document to its purpose so a reader knows where to continue.

4.4.12 Dark and high-contrast themes

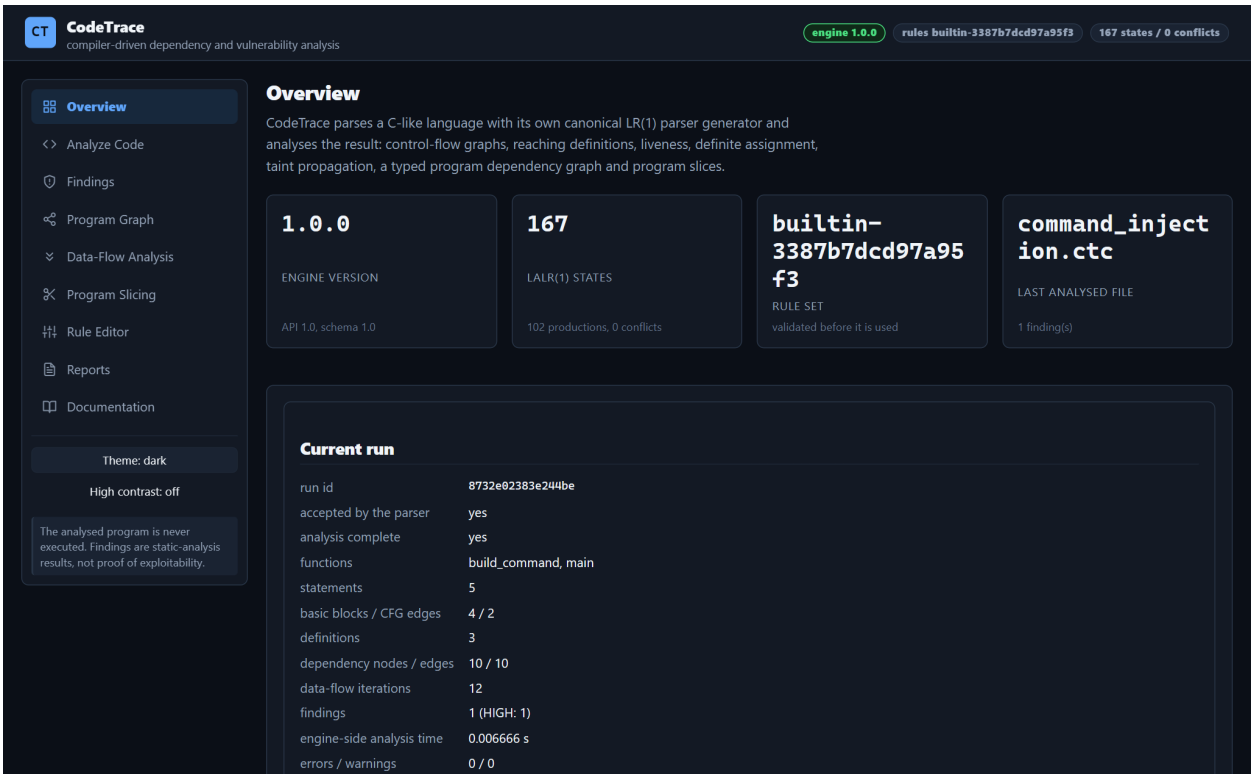


Figure 4.12: Dark and high-contrast themes — screenshot of the running application.

The whole workbench has a dark theme and a high-contrast mode. Contrast and focus styling are defined once, in the stylesheet, and every view inherits them; the theme choice is remembered for the session. The same run and the same summary are shown as in the light theme.

- Theme switching is a class on the document root, so no view needs to know about it.
- Graph nodes and table headers keep their borders and focus rings in both themes.
- The screenshot shows the identical run summary as the first screenshot in this section, which confirms that presentation is separate from analysis.

4.5 Worked Example: Command Injection

The following excerpt is the bundled command-injection example. It builds a command string from untrusted input in a helper function and passes the result to `system()`; comments and unrelated lines are omitted for clarity.

`examples/vulnerable/command_injection.ctc`

```
string build_command(string user_input) {
  return "ls " + user_input;
}
```

```
int main() {
  string name = input();
  string command = build_command(name);
  system(command);
  return 0;
}
```

The configured source rule identifies `input()` as a source of externally influenced data. The value flows into `name`, crosses the `build_command` boundary, is combined with a command prefix, and reaches argument zero of `system()`. The analyser reports a potential command-injection flow and supplies the steps that justify it. Its certainty is approximate because the interprocedural summary is context-insensitive; exploitability is not established. A safer design passes arguments separately to a fixed executable.

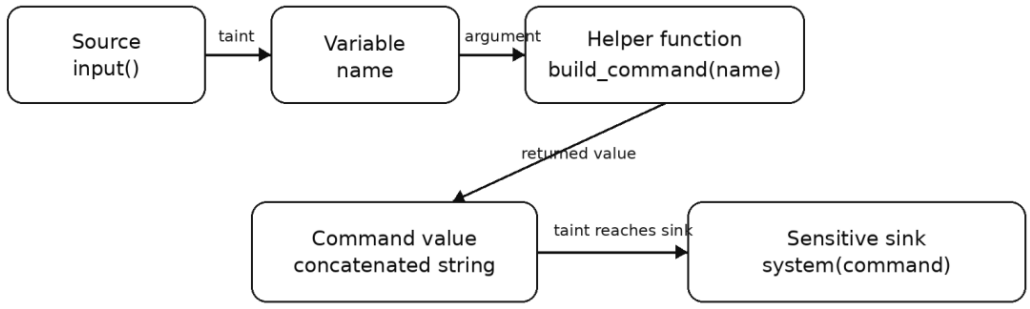


Figure 4.13: The reported evidence path for the worked example, from source to sink.

The same flow is reported by the interface as eight ordered steps. Each step names the statement, the symbol and the analysis fact that produced it, which is what makes the finding reviewable rather than assertive.

Table 4.4: The evidence path reported for the worked example.

#	Step	Location	What the step records
1	source	`main` 8:17	`input()` introduces untrusted data (source rule `stdin`).
2	declaration	`main` 8:3	`name` is assigned a tainted value.
3	argument	`main` 9:34	Argument 0 of `build_command()` is tainted.
4	parameter	`build_command` 3:22	`user_input` receives the value through the parameter list.
5	return	`build_command` 4:3	The returned expression carries the same taint atom.
6	call	`main` 9:20	The function summary propagates taint to the caller.
7	declaration	`main` 9:3	`command` is assigned the tainted result.
8	sink	`main` 10:10	`system()` argument 0 consumes data that is not safe for the `command` context.

Finding `FEF13A28F7E0E`, *rule* `command_execution`, *category* `command-injection`, *severity HIGH, certainty approximate. The run reports 43 tokens, 22 AST nodes, 5 statements, 4 basic blocks, 3 definitions and 10 dependency edges.*

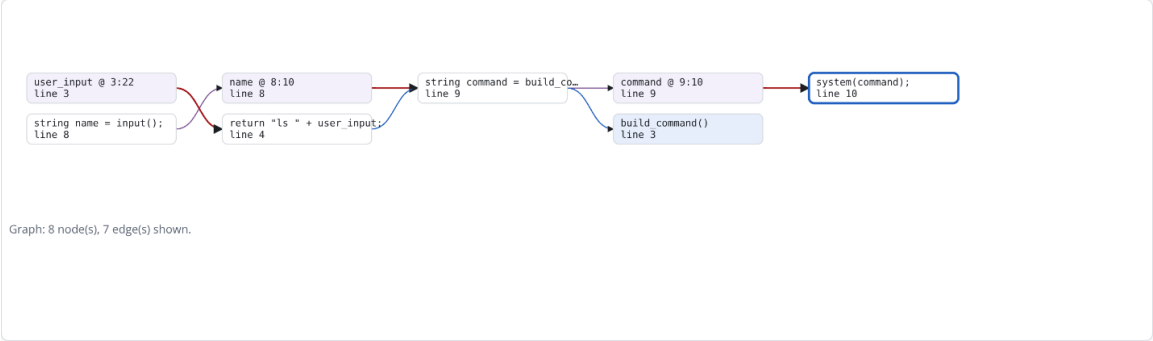


Figure 4.14: The dependency graph around the same flow, as published in the Review 2 material.

4.6 Located Diagnostics and Partial Analysis

The second kind of evidence is a refusal. Malformed input produces a located diagnostic and a partial-analysis status, and unsupported input is rejected with a code that names the construct. The table below lists representative cases from the bundled examples.

Table 4.5: Representative diagnostics from the bundled examples.

Example	Diagnostics	Interpretation
`syntax_errors/missing_semicolon.ctc`	`SYN001`	The grammar no longer matches; recovery keeps the rest of the file parseable and the run is marked partial.
`syntax_errors/unbalanced_braces.ctc`	`SYN001`, `SEM020`	The block structure is broken; the semantic stage reports the consequence rather than inventing a scope.
`unsupported/switch_statement.ctc`	`LEX100`, `LEX102`, `SYN100`	The construct is outside CT-C and is named as such instead of being silently ignored.
`unsupported/preprocessor_directive.ctc`	`LEX101`	Preprocessor semantics are outside the language model; the directive is reported.
`unsupported/pointer_type.ctc`	`SYN001`	Pointer types are not in the grammar; no pointer analysis is claimed.
`challenges/shadowed_variable.ctc`	`SEM018` (warning)	A shadowed declaration is legal but reported, because it changes which symbol a later statement refers to.
`challenges/array_element_taint.ctc`	`DFA001` (warning)	An element may be read before it is written; the array is not tracked element-wise.

4.7 Work File Explanation

The tables below explain every file in the repository. The purpose line is taken from the module's own docstring wherever one exists, so the map cannot drift from the code without a visible difference; the remaining lines describe data and asset files. Line counts are measured from this checkout and exclude the generated screenshot and PDF assets.

Compiler front end — `codetrace/frontend/`

Table 4.6: Work files in this group — 9 files, 3,277 lines (paths relative to `codetrace/frontend/`).

File	Lines	What it does
`__init__.py`	34	Front end for the CT-C source subset.
`ast_nodes.py`	431	Abstract syntax tree for the CT-C subset.
`clr_parser.py`	651	Table-driven LR parser: SLR(1), LALR(1) and canonical LR(1).
`grammar.py`	493	Grammar representation and LR analysis primitives.
`grammar_lang.py`	541	The CT-C grammar: productions, semantic actions and grammar documentation.
`lexer.py`	689	Hand-written lexer for the CT-C source subset.
`pipeline.py`	129	Lex + parse entry point.
`source.py`	43	Source-text helpers shared by the front end and the reporting layer.
`tokens.py`	266	Token definitions for the CodeTrace source language (CT-C).

Semantics — `codetrace/semantics/`

Table 4.7: Work files in this group — 4 files, 1,352 lines (paths relative to `codetrace/semantics/`).

File	Lines	What it does
`__init__.py`	0	package marker for semantic analysis
`analyzer.py`	913	Semantic analysis: declaration collection, scope resolution, type checking.
`externals.py`	196	Model of the host environment: known external functions.
`symbol_table.py`	243	Symbol tables with stable symbol identities.

Rules — `codetrace/rules/`

Table 4.8: Work files in this group — 4 files, 1,235 lines (paths relative to `codetrace/rules/`).

File	Lines	What it does
`__init__.py`	0	package marker for the rule model
`builtin/standard.ctrule`	209	the built-in rule set: 5 sources, 8 sinks, 5 sanitizers, 2 validators and 4 flows
`model.py`	305	Security rule model and registry.
`rule_language.py`	721	Grammar, parser and validator for the CodeTrace security rule language.

Analysis — `codetrace/analysis/`

Table 4.9: Work files in this group — 11 files, 4,130 lines (paths relative to `codetrace/analysis/`).

File	Lines	What it does
`__init__.py`	0	package marker for the analysis stages
`accesses.py`	153	Variable reads and writes per statement.
`cfg.py`	646	Control-flow graph construction over the CT-C AST.
`dataflow.py`	324	A small, reusable data-flow analysis framework.
`definite_assignment.py`	172	Definite-assignment analysis (the "possibly uninitialised variable" check).
`dependencies.py`	547	Program dependency graph: data, control, call, return, taint and sanitizer edges.
`liveness.py`	159	Live-variable analysis.
`pipeline.py`	536	Whole-program analysis pipeline.
`reaching_definitions.py`	185	Reaching-definitions analysis.
`slicing.py`	325	Program slicing over the program dependency graph.
`taint.py`	1,083	Taint analysis: sources, sinks, sanitizers, interprocedural propagation.

Findings and reporting — `codetrace/findings/`, `codetrace/reporting/`

Table 4.10: Work files in this group — 6 files, 1,875 lines (paths relative to `the repository root`).

File	Lines	What it does
`codetrace/findings/__init__.py`	0	package marker for the finding model
`codetrace/findings/explain.py`	1,026	Finding construction and path explanation.
`codetrace/findings/models.py`	199	Finding model.
`codetrace/reporting/__init__.py`	0	package marker for the report builders
`codetrace/reporting/html_report.py`	297	Human-readable HTML report.
`codetrace/reporting/json_report.py`	353	Analysis report: the documented, stable JSON structure CodeTrace exports.

HTTP API — `codetrace/api/`

Table 4.11: Work files in this group — 4 files, 1,152 lines (paths relative to `codetrace/api/`).

File	Lines	What it does
`__init__.py`	0	package marker for the HTTP layer
`__main__.py`	21	Run the CodeTrace HTTP API: python3 -m codetrace.api [options].
`schemas.py`	140	Request/response schemas of the CodeTrace HTTP API.
`server.py`	991	CodeTrace HTTP API (Python standard library only).

Package root — `codetrace/`

Table 4.12: Work files in this group — 2 files, 22 lines (paths relative to `the repository root`).

File	Lines	What it does
`codetrace/__init__.py`	0	package marker; the public version identifiers live in version.py
`codetrace/version.py`	22	Single source of truth for the CodeTrace version identifiers.

Interface — `frontend/`

Table 4.13: Work files in this group — 26 files, 6,446 lines (paths relative to `frontend/`).

File	Lines	What it does
`css/app.css`	1,308	all styling: layout, tables, graph canvas, light/dark/high-contrast themes
`index.html`	201	single-page shell: navigation, view containers and the analysis form
`js/api.js`	94	HTTP client: relative URLs, timeouts and structured ApiError values for the views
`js/app.js`	632	CodeTrace interface: routing, state wiring and view updates.
`js/graph.js`	176	SVG renderer for the program dependency graph, the CFG and slice views.
`js/lib/dom.js`	65	Minimal DOM helpers.
`js/lib/filter.js`	106	Filtering, searching and sorting of findings.
`js/lib/layout.js`	117	Deterministic layered layout for program graphs.
`js/source.js`	83	renders the analysed source with statement highlights and line mapping
`js/state.js`	106	Application state.
`js/views/analyze.js`	258	Analyze Code: submit source to the engine and show what came back.
`js/views/dataflow.js`	129	Data-Flow Analysis view: the CFG of one function with the results of the classical analyses, exactly as the engine computed them.
`js/views/docs.js`	86	Documentation view: the language subset, the diagnostic catalogue, the grammar of the analysed language and the project documents.
`js/views/findings.js`	272	Findings: the list, the filters and the evidence for one finding.
`js/views/graph.js`	286	Program Graph view: the dependency graph, a single CFG, or the subgraph of a finding / slice.
`js/views/overview.js`	107	Overview: what the tool is, what it just analysed, and how the pipeline fits together.
`js/views/reports.js`	51	Reports: export the JSON or HTML report of the current run, inspect the schema and compare two runs (new / resolved / persistent / moved findings).
`js/views/rules.js`	77	Rule Editor.
`js/views/slicing.js`	125	Program Slicing view.
`js/views/statement.js`	42	Selected-statement panel, shared by the graph, data-flow and slicing views.
`package.json`	10	test-only manifest; the application itself has no npm dependency
`tests/dom.js`	418	A deliberately small DOM implementation for the frontend tests.
`tests/harness.js`	150	Test harness: a real engine plus a stub DOM.
`tests/lib.test.js`	337	Unit tests for the frontend's pure modules: DOM helpers (escaping), finding filters, graph layout and the SVG renderer's edge selection.
`tests/regressions.test.js`	377	Regression tests for the interface defects confirmed by the hostile audit.
`tests/views.test.js`	833	Interface integration tests.

Interface tests — `frontend/tests/`

Table 4.14: Work files in this group — 5 files, 2,115 lines (paths relative to `frontend/tests/`).

File	Lines	What it does
`dom.js`	418	A deliberately small DOM implementation for the frontend tests.
`harness.js`	150	Test harness: a real engine plus a stub DOM.
`lib.test.js`	337	Unit tests for the frontend's pure modules: DOM helpers (escaping), finding filters, graph layout and the SVG renderer's edge selection.
`regressions.test.js`	377	Regression tests for the interface defects confirmed by the hostile audit.
`views.test.js`	833	Interface integration tests.

Engine tests — `tests/`

Table 4.15: Work files in this group — 18 files, 6,212 lines (paths relative to `tests/`).

File	Lines	What it does
`__init__.py`	1	CodeTrace test suite (Python standard library unittest).
`helpers.py`	69	Shared helpers for the test suite.
`test_api.py`	687	API tests against a real HTTP server.
`test_audit.py`	187	Tests for the audit script.
`test_audit_regressions.py`	668	Regression tests for compiler, HTTP API and interface defects.
`test_cfg.py`	394	Control-flow graph tests.
`test_clr_theory.py`	294	Parser-generator checks against textbook answers and an independent builder.
`test_core_battery.py`	123	Hand-labelled taint battery (positive, negative and boundary cases).
`test_dataflow.py`	708	Data-flow analysis tests.
`test_lexer.py`	124	Lexer tests: tokens, positions, literals, comments and every lexer diagnostic.
`test_mutation_check.py`	132	Tests for the mutation checker itself.
`test_parser.py`	300	Parser tests: LR(1) construction, parsing, diagnostics and recovery.
`test_reporting.py`	393	Reporting tests: the JSON report and the self-contained HTML report.
`test_rules.py`	353	Rule language tests: the declarative rule file is untrusted input.
`test_semantics.py`	164	Semantic analysis tests: scopes, shadowing, types, symbols and diagnostics.
`test_slicing.py`	289	Dependency graph and slicing tests.
`test_taint.py`	779	Taint analysis tests: the flows that must be found and the ones that must not.
`test_workflow.py`	547	End-to-end workflow tests over the real HTTP API.

Evaluation — `evaluation/`

Table 4.16: Work files in this group — 2 files, 754 lines (paths relative to `evaluation/`).

File	Lines	What it does
`expectations.json`	420	hand-written labels for the 33-file corpus, one block per case
`run_evaluation.py`	334	Evaluate CodeTrace against the labelled corpus in evaluation/expectations.json.

Scripts — `scripts/`

Table 4.17: Work files in this group — 11 files, 5,890 lines (paths relative to `scripts/`).

File	Lines	What it does
`audit.py`	1,007	Independent audit of a CodeTrace checkout.
`build_project_report.py`	775	Build docs/CodeTrace_Project_Report.pdf from its editable source.
`build_review2_docx.py`	1,004	Build the Review 2 project report as a .docx on the institutional template.
`capture_review2_screenshots.py`	144	
`evaluate.sh`	19	runs the labelled corpus and writes JSON evidence to var/
`generate_review2_pdf.py`	1,023	
`make_review2_figures.py`	219	Build the figure assets used by the Review 2 report.
`mutation_check.py`	401	Mutation check: does the test suite actually notice when the engine breaks?
`review2_content.py`	1,239	Report content for the Review 2 Word report.
`serve.sh`	18	thin wrapper around python3 -m codetrace.api
`tests.sh`	41	runs every suite: engine, API, reporting, workflow and interface

Examples — `examples/`

Table 4.18: Work files in this group — 34 files, 307 lines (paths relative to `examples/`).

File	Lines	What it does
challenges/array_element_taint.ctc	8	expects 1 finding: taint on an array element is attributed to the whole array.
challenges/decoy_in_comment_and_string.ctc	9	expects 1 finding only: the two decoys below are text, not calls.
challenges/guarded_and_checked.ctc	8	expects 1 finding, feasibility guarded: the sink is inside a branch that was not evaluated.
challenges/loop_carried.ctc	11	expects 1 finding: the tainted value flows through a loop-carried assignment.
challenges/multiple_sources_two_sinks.ctc	8	expects 2 findings: stdin and an environment variable reach two different sinks.
challenges/negated_validator_guard.ctc	8	expects 1 finding, feasibility guarded: the validator guards the other branch.
challenges/shadowed_variable.ctc	10	expects 1 finding: only the outer name is tainted; the inner one is a constant.
challenges/unknown_external_keeps_taint.ctc	7	expects 1 finding plus a SEM008 warning: an unknown function never clears taint.
challenges/wrong_context_sanitizer.ctc	8	expects 2 findings: sql_escape does not neutralise command context, and the raw value still reaches sql_exec.
custom_rules/custom_rule_demo.ctc	9	example: used with examples/custom_rules/strict_rules.ctrule: the custom sink is.
custom_rules/strict_rules.ctrule	36	example rule file used by the Rule Editor demonstration
interprocedural/argument_flow.ctc	15	expects 1 finding, rule command_execution: the taint crosses two call boundaries.
interprocedural/recursion.ctc	14	expects 1 finding, rule command_execution, certainty unproven (recursion limit reached).
interprocedural/return_value.ctc	11	expects 1 finding, rule command_execution, interprocedural true.
safe/constant_command.ctc	6	expects no findings, no diagnostics.
safe/dead_branch_sink.ctc	8	expects no findings: the sink sits in a branch whose condition is the literal 0.
safe/killed_by_constant.ctc	7	expects no finding: the tainted value is overwritten before the sink.
safe/source_never_reaches_sink.ctc	7	expects no findings: the source value is overwritten before the sink.
safe/unrelated_variable.ctc	8	expects no findings: the tainted value is printed, but a different value reaches the sink.
sanitized/parse_to_int.ctc	8	expects no findings: to_int destroys the text representation of the value.
sanitized/shell_escape.ctc	7	expects no findings: shell_escape neutralises command-context taint.
sanitized/sql_escape.ctc	7	expects no findings: sql_escape neutralises SQL-context taint.
sanitized/validated_command.ctc	8	expects no findings: the value is used only behind a declared validator.
syntax_errors/broken_expression.ctc	5	expects a syntax error inside the expression, reported at the failing token.
syntax_errors/missing_semicolon.ctc	6	expects exactly one SYN001 error with a line:column and an expected-token list.
syntax_errors/unbalanced_braces.ctc	5	expects a syntax error, and the parser reports where the construct is incomplete.
unsupported/pointer_type.ctc	6	expects syntax/unsupported diagnostics: pointer types are outside the analysed subset.
unsupported/preprocessor_directive.ctc	7	expects LEX101 preprocessor diagnostic; the directive is reported, not ignored.
unsupported/switch_statement.ctc	9	expects syntax/unsupported diagnostics; the file is not analysed as if switch were safe.
vulnerable/command_injection.ctc	12	expects 1 finding, rule command_execution, HIGH, category command-injection.
vulnerable/format_string.ctc	6	expects 1 finding, rule format_string, MEDIUM.

File	Lines	What it does
vulnerable/path_traversal.ctc	8	expects 1 finding, rule path_traversal, HIGH.
vulnerable/sql_injection.ctc	7	expects 1 finding, rule sql_injection, CRITICAL, category sql-injection.
vulnerable/two_step_command.ctc	8	expects 1 finding, rule command_execution, reached through three assignments.

Documentation — docs/

Table 4.19: Work files in this group — 13 files, 4,050 lines (paths relative to docs/).

File	Lines	What it does
ALGORITHMS.md	438	Algorithms, pseudocode and complexity — Every algorithm below is the one that runs in this checkout.
API.md	233	HTTP API — The engine and the browser talk over a small JSON API served by the same process
COMPILER_THEORY.md	901	Compiler theory in CodeTrace — This document explains every compiler concept the project is built on, why it is
CodeTrace_Project_Report.md	358	editable source of the BCSE307P project report: chapters, verification evidence and the report template commands
EVALUATION.md	247	Evaluation — CodeTrace ships a hand-labelled corpus and a harness that runs the real
LANGUAGE.md	288	The CT-C subset — CodeTrace analyses a deliberately small C-like language called CT-C subset
LIMITATIONS.md	177	Limitations and threat model — Two different questions are answered here: 1.
PRESENTATION.md	295	Presentation outline and viva questions — For presenting CodeTrace, and for the questions that usually follow.
REPORT_SCHEMA.md	154	Report schema — with codetrace/reporting/html_report.py.
REVIEW_2_REPORT.md	175	Review 2 deliverables: features added, work-file map, module reference and measured results
RULE_LANGUAGE.md	294	The CodeTrace rule language — CodeTrace does not hard-code "sources", "sinks" or "sanitizers" in Python.
TECHNICAL_DESIGN.md	249	Technical design — CodeTrace takes a single source file in the CT-C subset (docs/LANGUAGE.md),
USER_GUIDE.md	241	User guide — CodeTrace analyses one file written in the CT-C subset (docs/LANGUAGE.md) and

Repository root

Table 4.20: Work files in this group — 3 files, 204 lines (paths relative to the repository root).

File	Lines	What it does
README.md	169	project overview, quick start, repository layout, documentation map and how each result is verified
.gitignore	33	generated analysis output, caches and local scratch files are kept out of the tree
.gitattributes	2	line-ending normalisation for the shell scripts

4.8 Module-Wise Reference

This section is the reference the reader needs in order to check a claim quickly: for each module, the symbols it exposes, the invariant it maintains and the literature it implements. Reference numbers point to the bibliography at the end of the report.

4.8.1 Front end and semantic analysis

Table 4.21: Front-end and semantic modules.

Module	Key symbols	Invariant and output	Ref.
`frontend/lexer.py`	`Lexer`, `tokenize`, `Diagnostic`, `LexResult`	Every token has a kind, a value, a byte span and a `line:column`; the scanner always advances, and every lexical error carries a code.	[1], [2]
`frontend/tokens.py`	`Token`, `display_kind`	Token kinds are stable strings, so diagnostics and tests can refer to them.	[1]
`frontend/grammar.py`	`Grammar`, `Production`, `Automaton`, `build_automaton`	CLOSURE, GOTO, FIRST/FOLLOW, canonical LR(1) item sets and LALR state merging over one augmented grammar.	[1], [2], [3], [11]
`frontend/clr_parser.py`	`build_tables`, `ParserTables`, `LRParser`, `Action`	One table row per state; a conflict is recorded, never hidden; recovery keeps the longest viable prefix.	[2], [3], [11]
`frontend/grammar_lang.py`	`LanguageGrammarBuilder`, `grammar_documentation`	The CT-C grammar is defined once and documented from the same description.	[1]
`frontend/ast_nodes.py`	`Node` and one class per construct	Every node carries a half-open source range; traversal order is deterministic.	[1]
`frontend/pipeline.py`	`parse_source`, `parser_tables`, `ParseOutcome`	Tables are built once per mode and cached; the parse outcome carries diagnostics and an optional tree.	[2]
`frontend/source.py`	`LineIndex`, `split_lines`	Byte offsets and line/column positions are converted without re-scanning the file.	—
`semantics/symbol_table.py`	`Symbol`, `Scope`, `SymbolTable`, `Reference`	Each declaration gets a stable integer id; scopes nest and shadowing is explicit.	[1], [5]
`semantics/analyzer.py`	`SemanticAnalyzer`, `analyse`, `SemanticResult`	Names resolve to symbols, not to strings; per-statement reads and writes are recorded for the later stages.	[1], [5]
`semantics/externals.py`	`ExternalFunction`, `lookup`, `catalog_json`	The external-function catalogue is data, so a rule and a semantic check agree on arities.	[1]

4.8.2 Control flow, data flow and taint analysis

Table 4.22: Analysis modules.

Module	Key symbols	Invariant and output	Ref.
`analysis/cfg.py`	`CFG`, `BasicBlock`, `CFGEdge`, `Definition`, `build_cfgs`	Blocks are maximal straight-line sequences; edges are typed; definitions are numbered once and reused by every data-flow problem.	[4], [5]
`analysis/accesses.py`	`statement_accesses`, `expression_accesses`	Reads and writes for a statement are computed once, so GEN/KILL and USE/DEF agree.	[5]
`analysis/dataflow.py`	`DataFlowProblem`, `BitVectorDomain`, `solve`	One monotone worklist solver with documented iteration and step caps; IN/OUT sets are bit vectors over a fixed domain.	[4], [5]
`analysis/reaching_definitions.py`	`ReachingDefinitions`, `definition_atom`	Forward may-analysis; the answer to 'which definitions may reach this statement'.	[4], [5]
`analysis/liveness.py`	`Liveness`, `symbol_atom`	Backward may-analysis; the answer to 'which values may be used after this point'.	[4], [5]
`analysis/definite_assignment.py`	`DefiniteAssignment`	Forward must-analysis; an uninitialised read is reported as `DFA001`, never assumed away.	[4], [5]
`analysis/taint.py`	`TaintEngine`, `TaintProblem`, `SinkHit`, `TaintResult`	One taint atom per context; sanitizers remove only their own context; summaries are iterated to a fixed point and their approximations are recorded.	[9], [10], [13]
`analysis/dependencies.py`	`DependencyGraph`, `DepEdge`, `DepNode`	Every edge carries the analysis fact that justifies it; no edge is added without a reason string.	[7], [12]
`analysis/slicing.py`	`SliceEngine`, `Slice`	Slices are reachability over the dependency graph; each included statement carries a reason.	[6], [7], [12]
`analysis/pipeline.py`	`analyse_source`, `Program`, `FunctionAnalyses`	Stages run in a fixed order and share one `Program`; a stage that cannot complete records a partial reason instead of raising.	[4], [9]

4.8.3 Rules, findings, reporting and the interface

Table 4.23: Rule, finding, reporting and interface modules.

Module	Key symbols	Invariant and output	Ref.
`rules/model.py`	`RuleSet`, `SourceRule`, `SinkRule`, `SanitizerRule`, `FlowRule`	A rule set is immutable data with a fingerprint; `stats()` is what the interface displays.	[9], [10]
`rules/rule_language.py`	`parse_rule_text`, `validate_declarations`, `load_builtin_rules`	Rules are parsed by the project's own LR parser and validated with `line:col` diagnostics before use.	[1], [2], [9]
`rules/builtin/standard.ctrule`	the built-in rules	5 sources, 8 sinks, 5 sanitizers, 2 validators and 4 flows, all visible and editable.	[9], [10]
`findings/models.py`	`Finding`, `stable_finding_id`, `compare_findings`	Identity depends on the rule, the sink operation and the statement text, not on the line number.	[9]
`findings/explain.py`	`PathExplainer`, `build_findings`, `TraceStep`	Each finding carries an ordered path, branch conditions, certainty, limitations and remediation.	[6], [9]
`reporting/json_report.py`	`build_report`, `write_json_report`, `dispositions`	The report follows the documented schema; validator suppressions are reported, not silently applied.	—
`reporting/html_report.py`	`render_html`, `write_html_report`	The HTML report is one file, with no script and with every analysed text escaped.	—
`api/server.py`	`CodeTraceHandler`, `RunStore`, `create_server`	Requests are validated before the engine runs; limits and error codes are documented; runs live in memory only.	—
`api/schemas.py`	the schema catalogue	Schemas are data, so the documentation endpoint and the tests read the same description.	—
`frontend/js/app.js`	router, run lifecycle, theme	One run identifier is shared by every view; stale text is flagged before results are read.	—
`frontend/js/state.js`	the store	Only the store holds analysis results, so views cannot disagree about the run.	—
`frontend/js/graph.js`, `js/lib/layout.js`	renderer and layered layout	Deterministic positions for one graph; nodes are focusable and selectable.	—
`frontend/js/views/`	the nine views	A view renders from state and API data only; it never re-derives an analysis fact.	—

4.9 Verification, Testing and Results

Every claim in this report is produced by a command in the repository. The engine suite is run with the standard library's test runner, the interface suite with the Node test runner against a real engine, the corpus evaluation over the labelled examples, and the audit checks against the files in the tree.

Table 4.24: The verification instruments and their results on this checkout.

Instrument	Command	Result
Engine and integration suite	`python3 -m unittest discover -s tests -t .`	350 tests, 350 passed
Interface suite	`npm test` in `frontend/`	53 tests, 53 passed
Independent audit	`python3 scripts/audit.py`	69 checks, 69 passed
Mutation check	`python3 scripts/mutation_check.py`	18 of 18 defects caught
Labelled corpus	`python3 evaluation/run_evaluation.py`	33 files, 1.26 s, precision 1.0, recall 1.0

The corpus contains 20 positive cases, 9 files that must stay silent and 4 cases that must be rejected before analysis, so precision and recall are measured against hand-written labels rather than against the tool's own output. The audit is deliberately independent of the test suite: it re-reads the documents, the example expectations and the module table, and fails if a quoted number no longer matches the tree. The mutation check edits the engine in 18 small, plausible ways, one at a time, and requires the suite to notice each edit; a surviving mutation is treated as a hole in the tests.

Table 4.25: Corpus results by group.

Group	Files	Expected	Reported	Outcome
Vulnerable flows	18	20 findings	20 findings	all matched
Safe and sanitized programs	9	no findings	no findings	all silent
Rejected input	6	diagnostics only	diagnostics only	no finding claimed
Total	**33**	20 findings	20 findings	precision 1.0, recall 1.0, F1 1.0

The corpus is authored by the project team and is therefore an internal measurement; independent external evaluation remains future work.

Current progress and challenges. CodeTrace connects CT-C parsing and semantic checks to control-flow, data-flow, taint, dependency, slicing and reporting modules, and the workbench exposes all of them over one run. A central design challenge is preserving source locations and symbol identities across the stages so that each result remains traceable to the relevant statements; a second is keeping the analysis honest about what it does not know, which is why partial runs, unsupported constructs and unreported sink calls are all visible.

The main limitations are the small language subset and the conservative analysis: pointers, alias analysis, preprocessor semantics and multi-file programs are outside scope; arrays are not tracked element-wise; implicit flows are not modelled; and interprocedural summaries are context-insensitive. Validator rules depend on declared assumptions, so findings can remain partial or approximate. These limits are stated per finding, in the run-level summary and in `docs/LIMITATIONS.md`, rather than left to be discovered.

Conclusion and future enhancements. CodeTrace demonstrates how compiler representations support explainable static program and security analysis. It connects grammar-based parsing and semantic analysis to control-flow and data-flow reasoning, then presents dependency and taint relationships through findings, slices and reports. Its contribution is inspectability within a documented language subset, not complete C support or proof of exploitability. Future work should prioritise context-sensitive function summaries, stronger branch reasoning, finer-grained array and alias modelling, broader language coverage, independent external evaluation and additional report formats.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[2] D. E. Knuth, “On the translation of languages from left to right,” Information and Control, vol. 8, no. 6, pp. 607–639, 1965.

[3] F. L. DeRemer, “Simple LR(k) grammars,” Communications of the ACM, vol. 14, no. 7, pp. 453–460, 1971.

[4] G. A. Kildall, “A unified approach to global program optimization,” in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1973, pp. 194–206.

[5] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Berlin, Germany: Springer, 1999.

[6] M. Weiser, “Program slicing,” IEEE Transactions on Software Engineering, vol. SE-10, no. 4, pp. 352–357, 1984.

[7] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” ACM Transactions on Programming Languages and Systems, vol. 9, no. 3, pp. 319–349, 1987.

[8] T. Reps, S. Horwitz, and M. Sagiv, “Precise interprocedural dataflow analysis via graph reachability,” in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1995, pp. 49–61.

[9] B. Chess and G. McGraw, “Static analysis for security,” IEEE Security & Privacy, vol. 2, no. 6, pp. 76–79, 2004.

[10] MITRE, “Common Weakness Enumeration (CWE),” including CWE-22, CWE-78, CWE-89 and CWE-134. [Online]. Available: https://cwe.mitre.org/

[11] F. DeRemer and T. Pennello, “Efficient computation of LALR(1) look-ahead sets,” ACM Transactions on Programming Languages and Systems, vol. 4, no. 4, pp. 615–649, 1982.

[12] S. Horwitz, T. Reps, and D. Binkley, “Interprocedural slicing using dependence graphs,” ACM Transactions on Programming Languages and Systems, vol. 12, no. 1, pp. 26–60, 1990.

[13] V. B. Livshits and M. S. Lam, “Finding security vulnerabilities in Java applications with static analysis,” in Proceedings of the 14th USENIX Security Symposium, 2005, pp. 271–286.

[14] OWASP Foundation, “OWASP Top 10:2021 — A03:2021 Injection.” [Online]. Available: https://owasp.org/Top10/